COMPUTER ORGANIZATION

MEMORY

Objectives:

- Master the concepts of hierarchical memory organization
- Understand how each level of memory contributes to system performance, and how the performance is measured
- Master the concepts behind cache memory, virtual memory, memory segmentation, paging, and address translation

Main Memory

RAM
(Random Access Memory)

ROM
(Read Only Memory)

RAM TYPES?

DRAM
(DYNAMIC RAM)

SROM
(STATIC RAM)

- DRAM slowly leak their charge over time
- DRAM bits must be refreshed every few milliseconds to prevent data loss
- DRAM is “cheap” memory

SRAM

- SRAM consists of circuits similar to the D flip-flop
- SRAM is very fast memory, and it doesn’t need to be refreshed like as DRAM does

Cache Memory

- The purpose of cache memory is to speed up accesses by storing recently used data closer to the CPU, instead of storing it in main memory
- Cache is much smaller than main memory
- Cash memory access time is a fraction of that of main memory

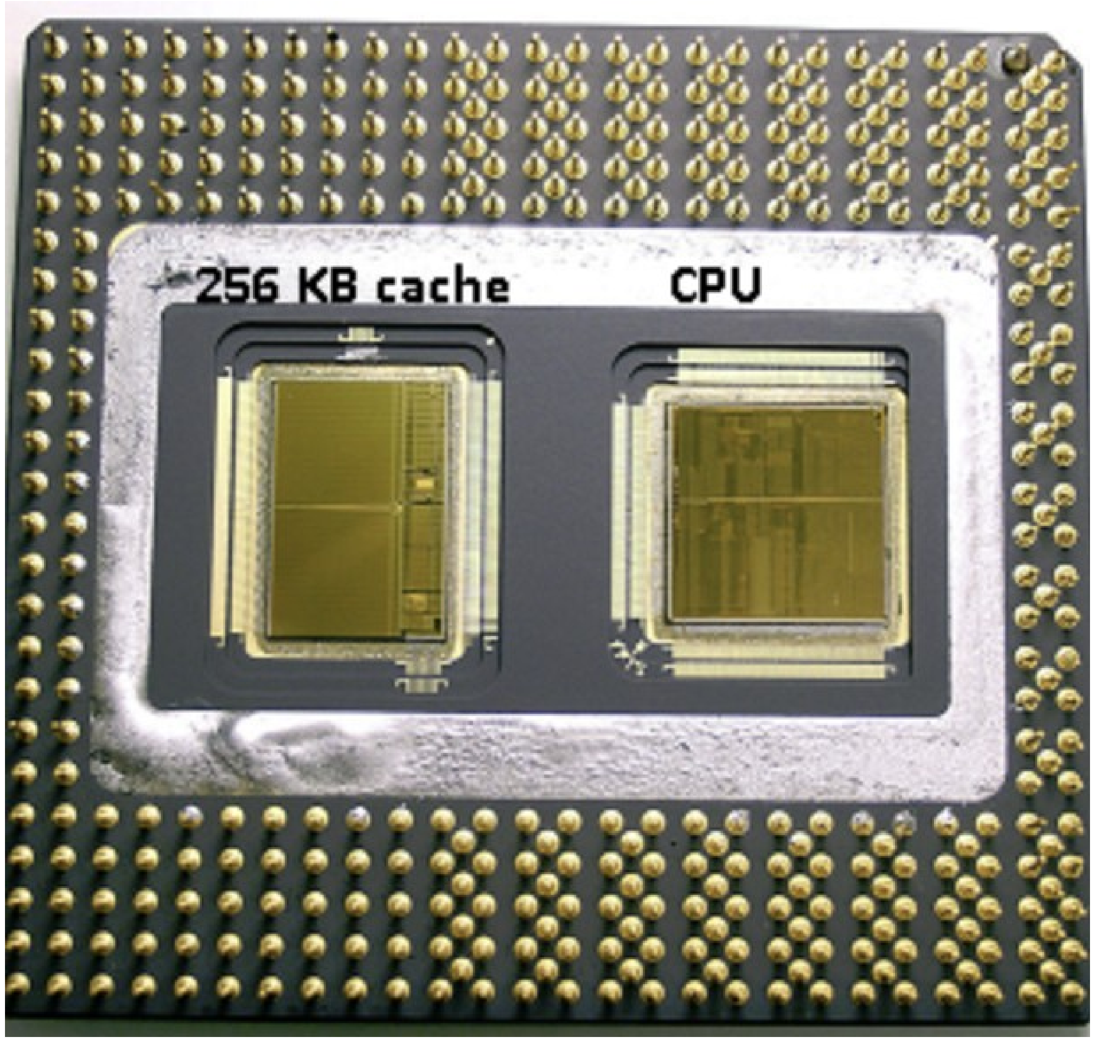
Cache Memory

- Cache memory is a component that makes retrieving data from the computer's memory more efficient
- The purpose of cache is to speed up memory accesses by storing recently used data closer to the CPU
- It acts as a temporary storage area that the computer's processor can retrieve data from easily

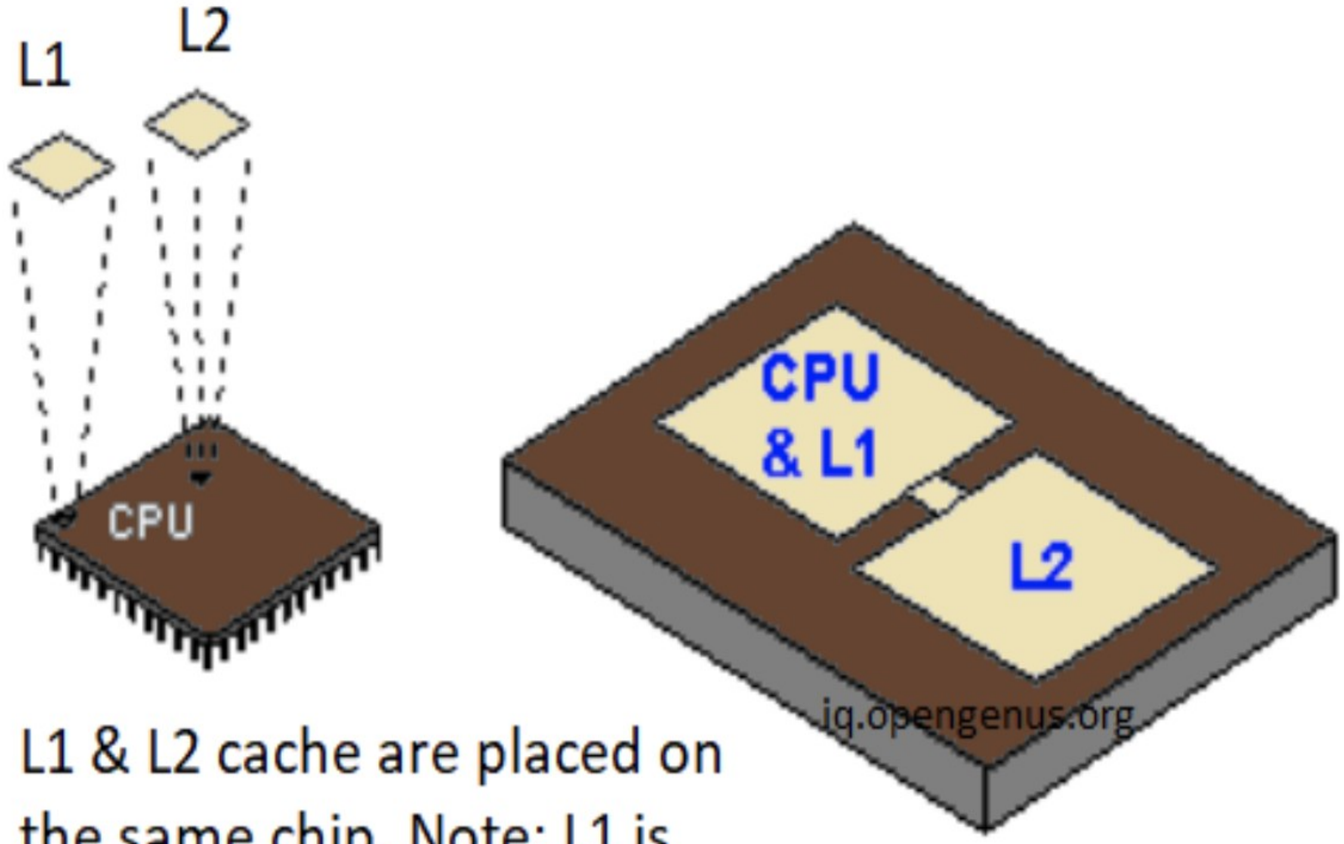
Cache Memory Levels

- **L1 cache**, or primary cache, is extremely fast but relatively small, and is usually embedded in the processor chip as CPU cache
- **L2 cache**, or secondary cache, is often more capacious than L1
- **Level 3 (L3) cache** is specialized memory developed to improve the performance of L1 and L2

L1 cache memory (16KB – 64KB)



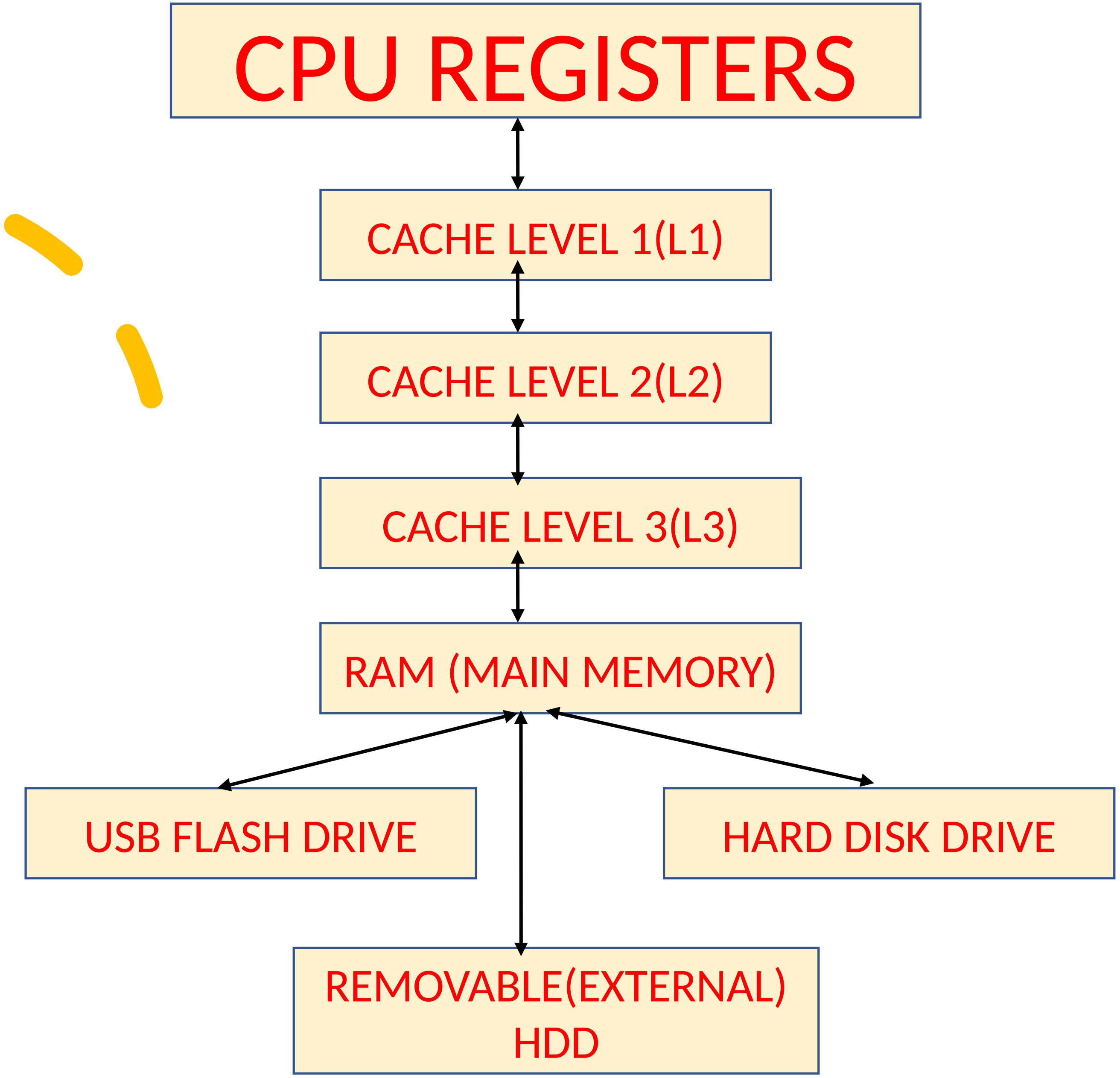
L2 cache memory (128KB – 8MB)



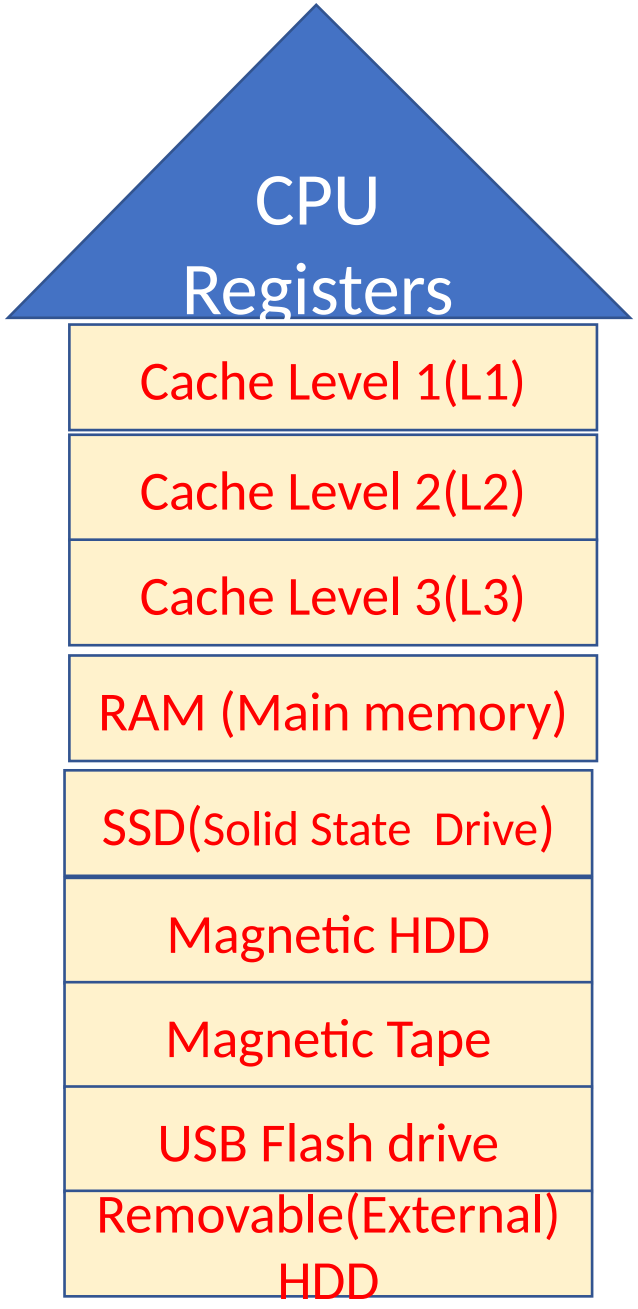
L1 & L2 cache are placed on the same chip. Note: L1 is placed in the CPU slot while L2 cache is next to it.

Figure: L1 & L2 cache on modern CPU

THE MEMORY DATA TRANSFER RELATIONSHIP



THE MEMORY HIERARCHY



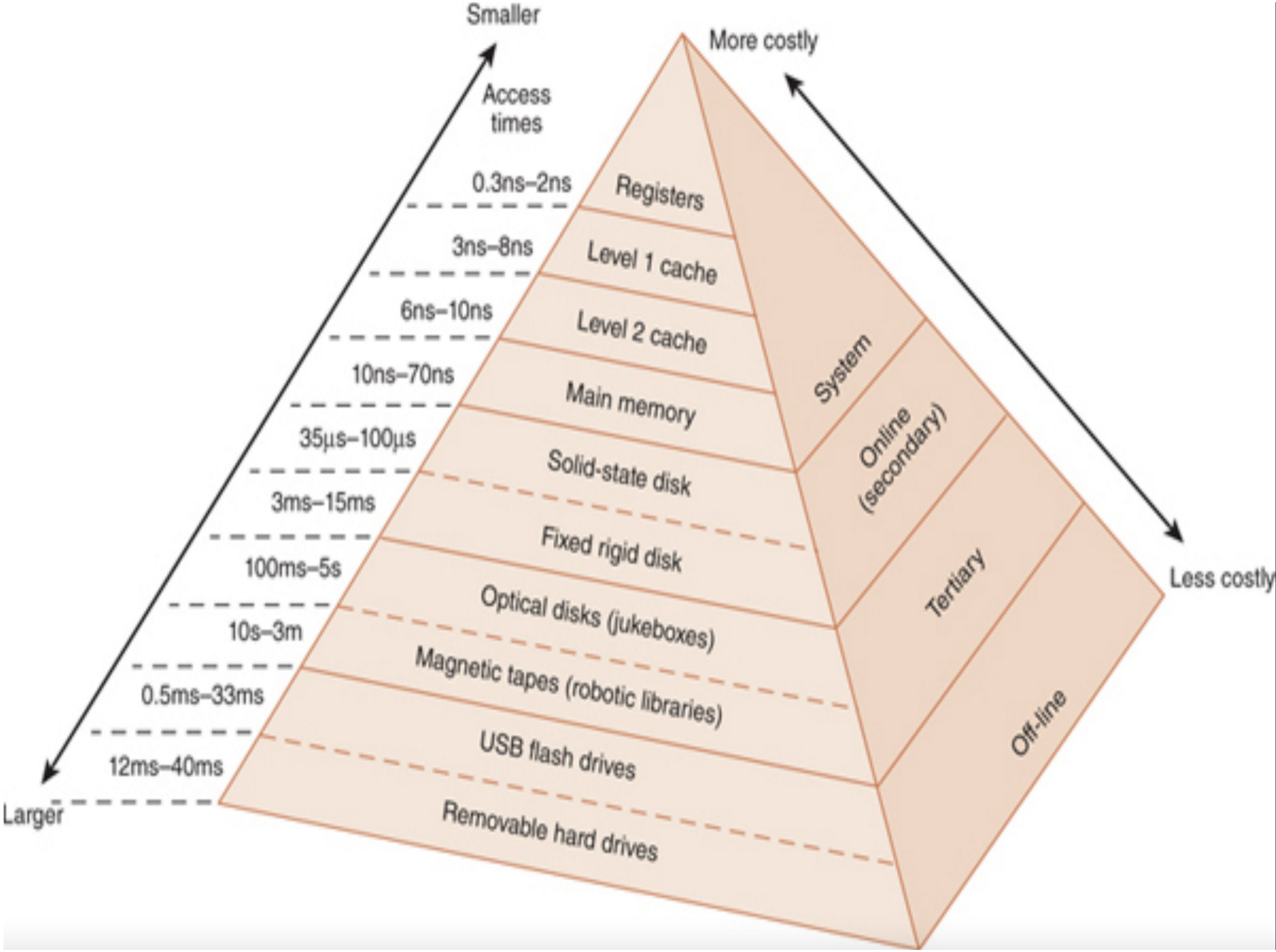
The Memory Hierarchy

- Generally speaking, faster memory is more expensive than slower memory.
- To provide the best performance at the lowest cost, memory is organized in a hierarchical fashion.
- Small, fast storage elements are kept in the CPU; larger, slower main memory is accessed through the data bus.
- Larger, (almost) permanent storage in the form of disk and tape drives is still further from the CPU.

The Memory Hierarchy

- To access a particular piece of data, the CPU first sends a request to its nearest memory, usually cache
- If the data is not in cache, then main memory is queried. If the data is not in main memory, then the request goes to disk
- Once the data is located, then the data and a number of its nearby data elements are fetched into cache memory

THE MEMORY HIERARCHY



Cache memory mapping

Direct mapped cache

Fully associative cache mapping

Set associative cache mapping

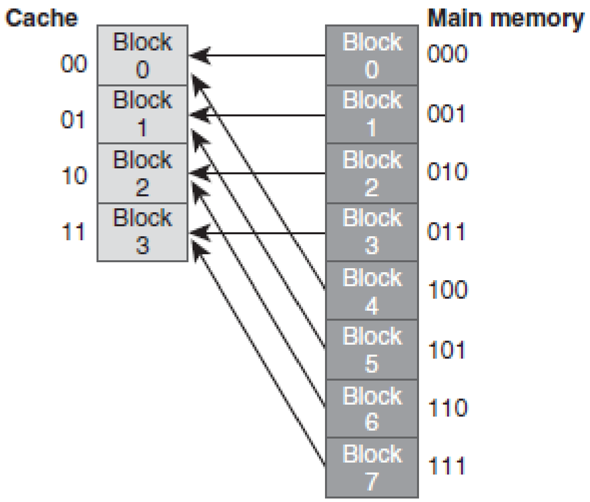
Direct-mapped cache

- The simplest cache mapping scheme is a direct-mapped cache
- In a direct-mapped cache consisting of N blocks of cache, block X of main memory maps to cache block

  $Y = X \bmod N$.
- Thus, if we have 10 blocks of cache, block 7 of the cache may hold blocks 7, 17, 27, 37, . . . of main memory.

Direct-mapped cache

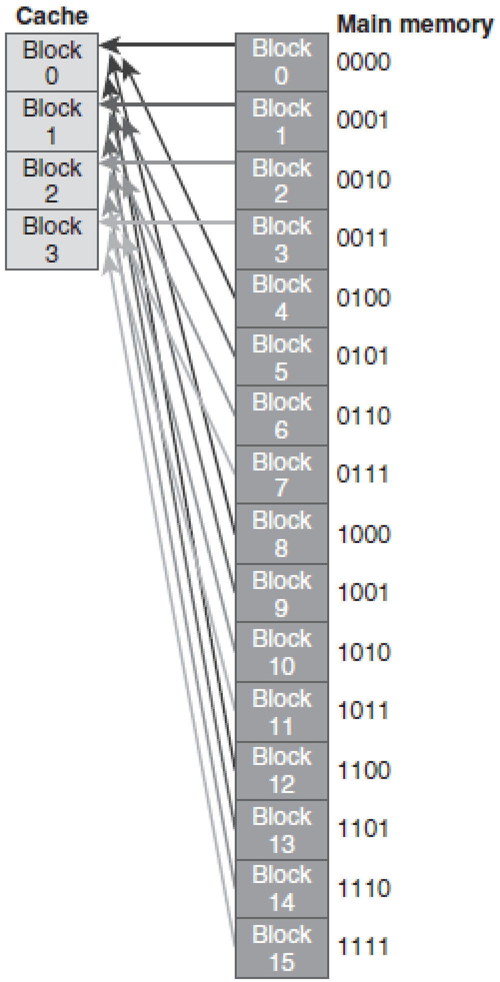
- With a direct-mapped cache consisting of 4 blocks of cache, block X of main memory maps to cache block Y = X mod 4.



8 memory blocks to 4 cache blocks

Direct-mapped cache

- A larger example



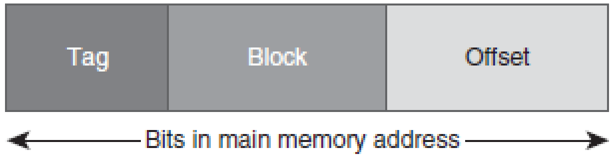
16 memory blocks to 4 cache blocks

DIRECT MEMORY MAPPING

- To perform direct mapping, the binary main memory address is partitioned into the fields shown below.

DIRECT MEMORY MAPPING

- **The offset field:**
  - uniquely identifies an address within a specific block
- **The block field**
  - selects a unique block of cache
- **The tag field:**
  - Tag bits identify the Main memory block residing in the cache line
  - is whatever is left over



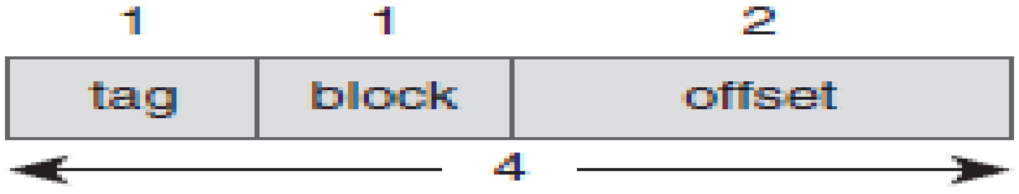
- The sizes of these fields are determined by characteristics of both memory and cache.

DIRECT MEMORY MAPPING

- Consider a byte-addressable main memory consisting of 4 blocks, and a cache with 2 blocks, where each block is 4 bytes.
- This means blocks 0 and 2 of main memory map to Blockblock 0 of cache, and Blocksblocks 1 and 3 of main memory map to block 1 of cache.
- Using the tag, block, and offset fields, we can see how main memory maps to cache as follows.

DIRECT MEMORY MAPPING

- Example 6.1: Cont’d. Consider a byte-addressable main memory consisting of 4 blocks, and a cache with 2 blocks, where each block is 4 bytes.
  - First, we need to determine the address format for mapping. Each block is 4 bytes, so the offset field must contain 2 bits; there are 2 blocks in cache, so the block field must contain 1 bit; this leaves 1 bit for the tag (as a main memory address has 4 bits because there are a total of 24 = 16 bytes).

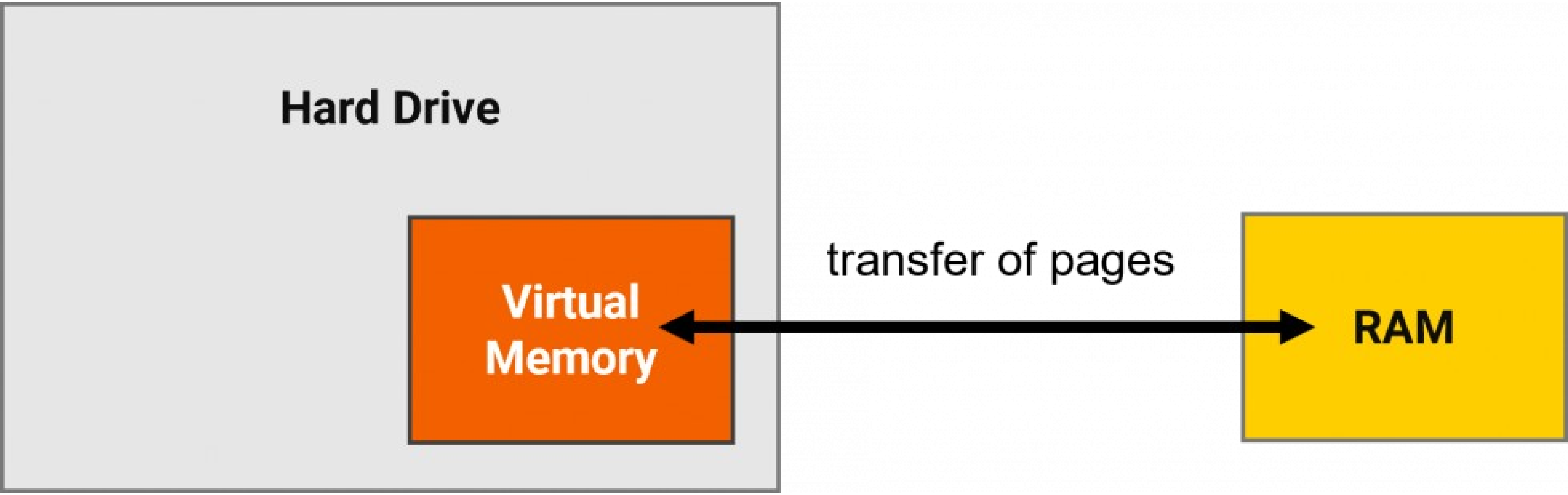


Main memory format

Virtual Memory?

- Virtual memory is **a memory management technique where secondary memory can be used as if it were a part of the** main memory
- Cache memory enhances performance by providing **faster memory access speed**
- Virtual memory enhances performance by providing **greater memory capacity**, without the expense of adding main memory.
- Instead, a portion of a disk drive serves as an extension of main memory
- If a system uses paging, virtual memory partitions main memory into individually managed page frames, that are written (or paged) to disk when they are not immediately needed

Virtual Memory?



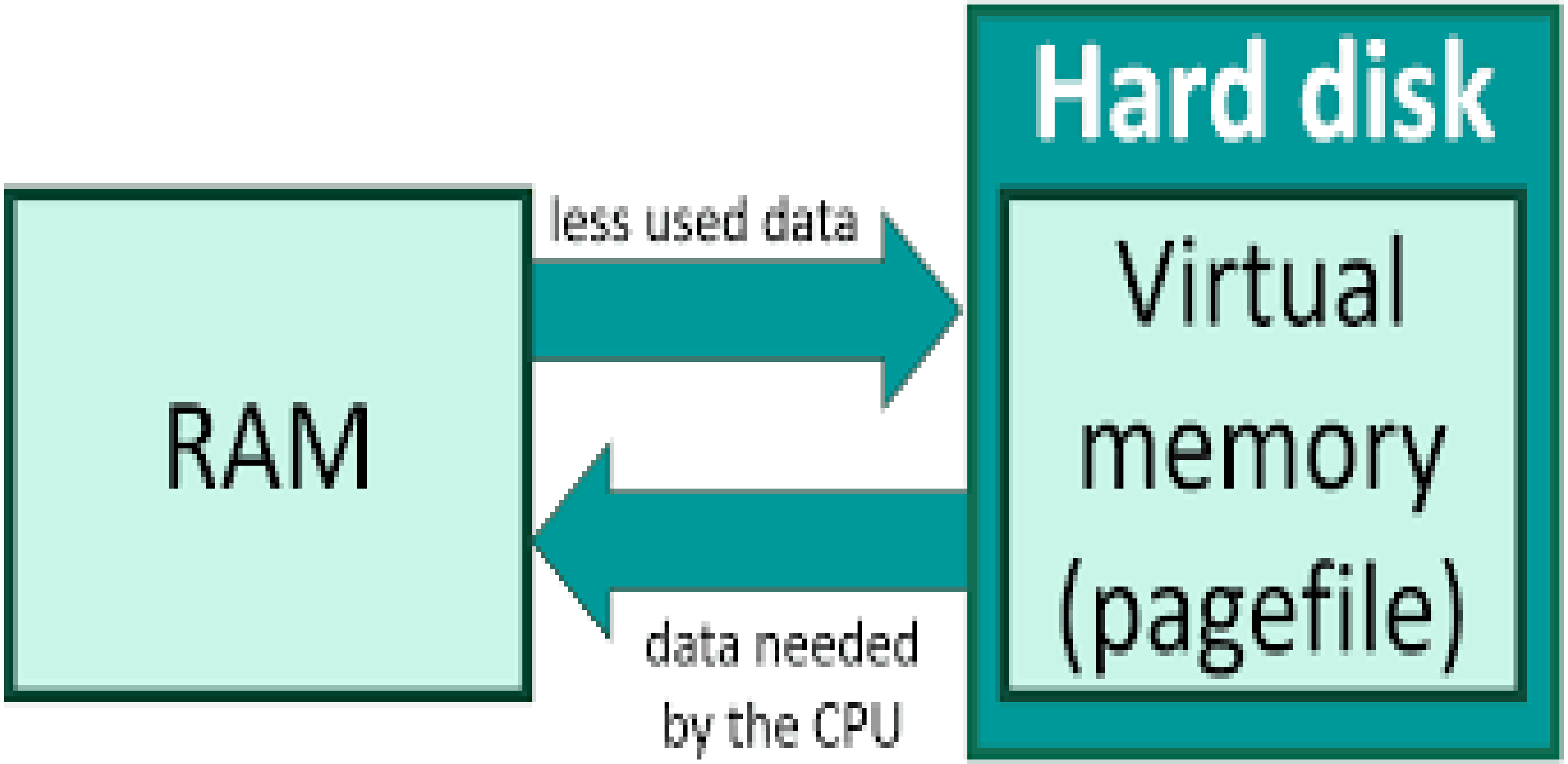
Virtual Memory

- *Virtual memory speeds up the execution of heavier applications without running out of memory*

- The virtual memory swaps unused data from RAM to a storage device like a hard drive or solid-state drive (SSD) and frees up RAM for other tasks

- Virtual memory is defined as a memory management method where computers use secondary memory to compensate for the shortage of physical memory

Virtual Memory?



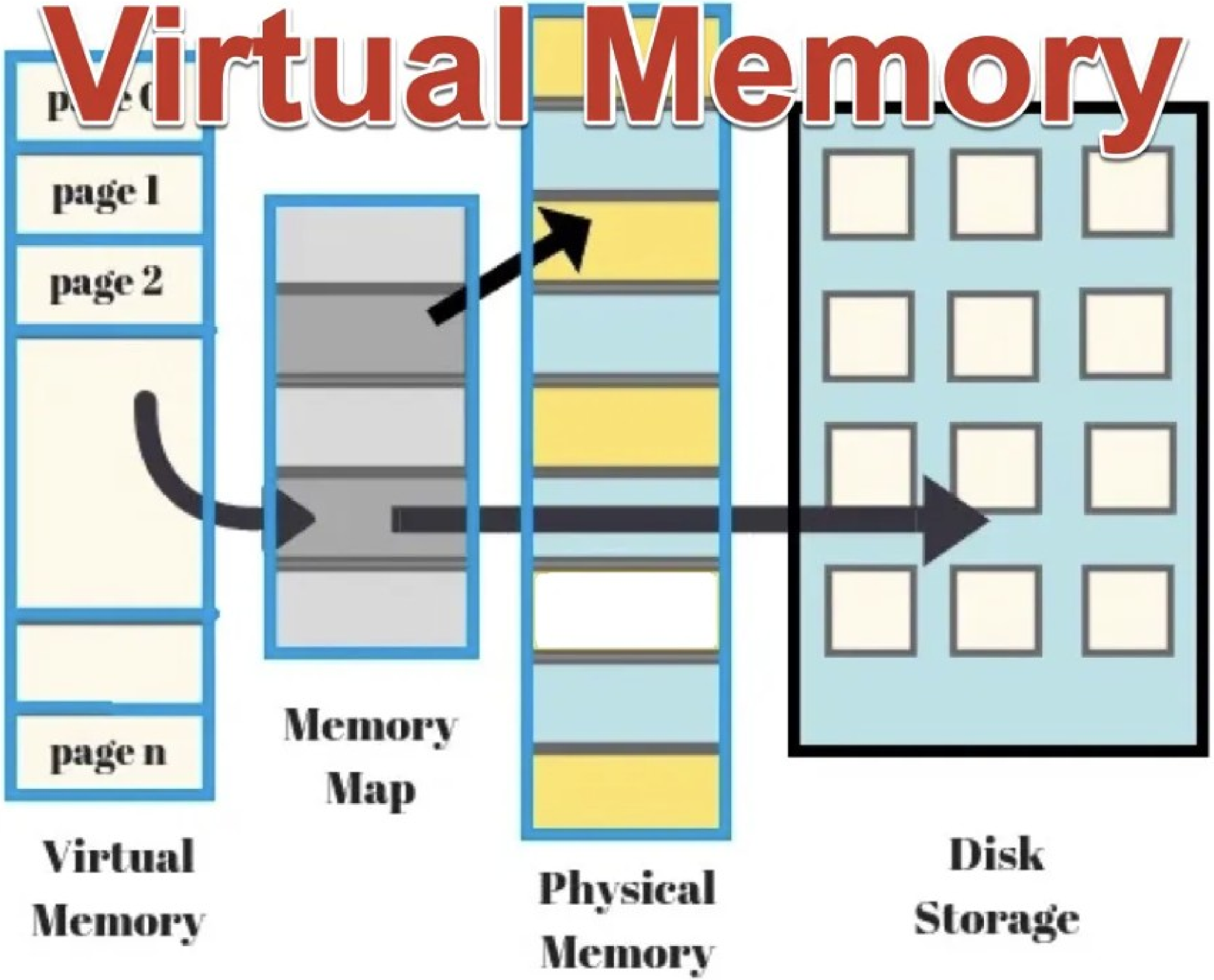
Virtual Memory

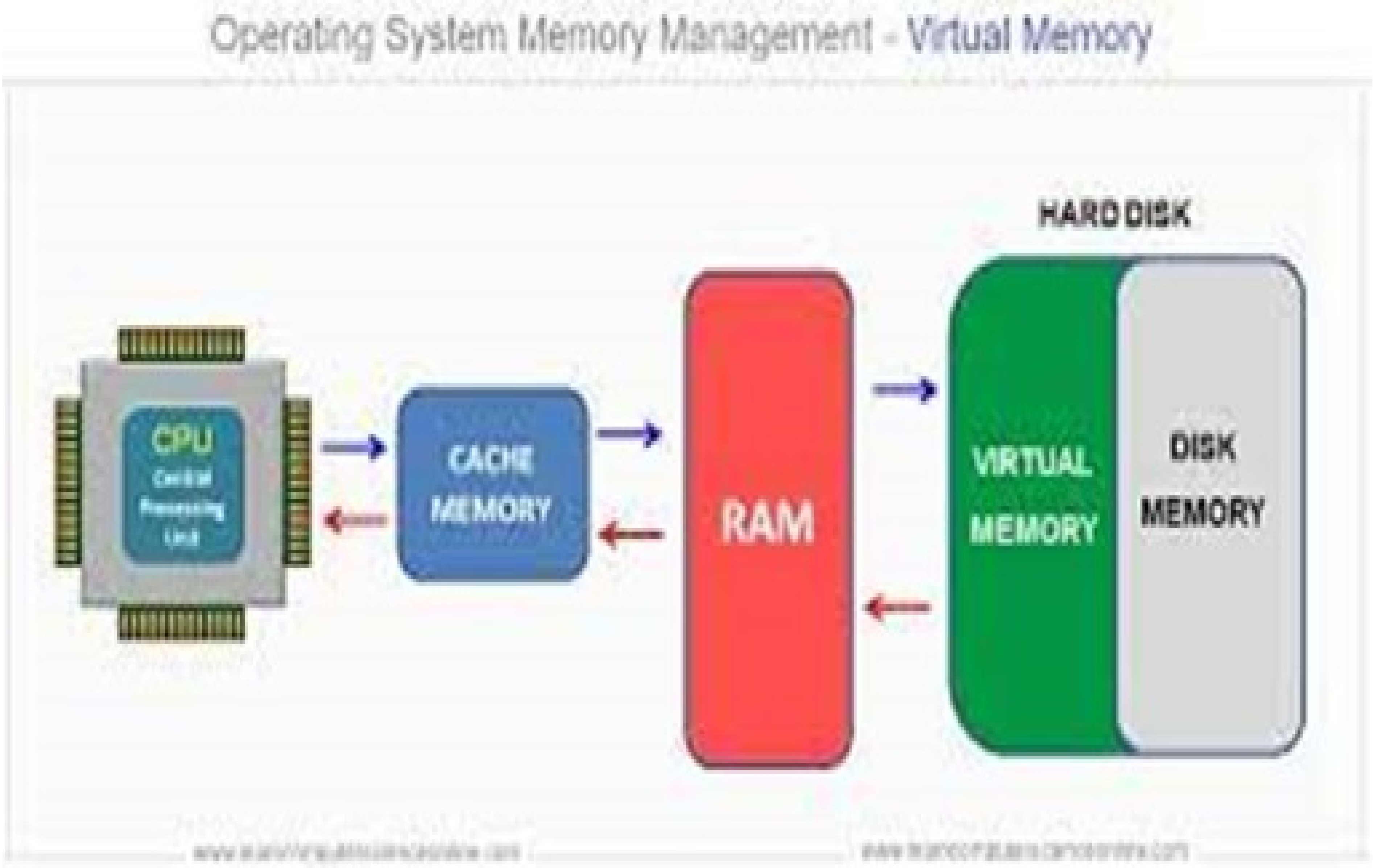
- Main memory and virtual memory are divided into equal-sized pages
- The entire address space required by a process need not be in memory at once. Some parts can be on disk, while others are in main memory
- Further, the pages allocated to a process do not need to be stored contiguously—either on disk or in memory.
- In this way, only the needed pages are in memory at any time, the unnecessary pages are in slower disk storage.

Benefits of Virtual Memory

- Balance the shortage of physical space
- Inexpensive virtual memory
- Supports multitasking and collaboration
- Avoid memory fragmentation
- Enhance data security

Virtual Memory




Operating System Memory Management - Virtual Memory
HARD DISK
CPU
Central Processing Unit
CACHE MEMORY
RAM
VIRTUAL MEMORY
DISK MEMORY